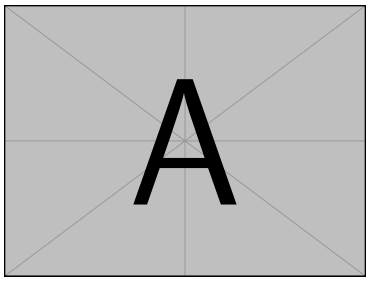
DATA ANALYTICS FUNDAMENTALS

Module 1: Comprehensive Course Notes



Dr. Lumumba W. Victor

Center for Data Analytics and Modeling (CDAM)
Chuka University

24th March 2026

Resource: `https://cdam.chuka.ac.ke`

Contents

Preface **5**

1 Basic R Programming Skills **6**
1.1 Introduction to R . . . 6
1.2 RStudio Interface . . . 6
1.2.1 Main Panes . . . 6
1.3 Data Types and Structures . . . 7
1.3.1 Basic Data Types . . . 7
1.3.2 Data Structures . . . 7
1.3.3 Matrices and Arrays . . . 8
1.3.4 Data Frames . . . 8
1.3.5 Lists . . . 10
1.4 Control Structures . . . 10
1.4.1 Conditional Statements . . . 11
1.4.2 Loops . . . 12
1.5 Functions . . . 12
1.6 Apply Family Functions . . . 14
1.7 Error Handling and Debugging . . . 15
1.8 Practice Exercises . . . 15

2 Data Importation and Cleaning **16**
2.1 Overview . . . 16
2.2 Importing Data from Various Sources . . . 16
2.2.1 CSV and Text Files . . . 16
2.2.2 Excel Files . . . 17
2.2.3 Database Connections . . . 18
2.2.4 Web Data and APIs . . . 19
2.3 Data Quality Assessment . . . 19
2.3.1 Identifying Data Quality Issues . . . 19
2.4 Handling Missing Data . . . 20
2.4.1 Types of Missing Data . . . 20
2.5 Data Cleaning Techniques . . . 21
2.5.1 String Cleaning . . . 22
2.5.2 Date and Time Cleaning . . . 23
2.5.3 Outlier Detection and Treatment . . . 23
2.6 Data Validation . . . 25
2.7 Data Export . . . 26
2.8 Practice Exercises . . . 26

3 Data Manipulation and Wrangling **27**
3.1 Introduction to dplyr . . . 27
3.2 Filtering Rows . . . 28
3.3 Selecting Columns . . . 29
3.4 Creating and Modifying Columns . . . 30
3.5 Summarizing Data . . . 30
3.6 Grouped Operations . . . 32
3.7 The Pipe Operator . . . 32

3.8 Joining Data Frames . . . 33
3.8.1 Types of Joins . . . 33
3.9 Reshaping Data . . . 35
3.10 String and Date Manipulation . . . 36
3.11 Practice Exercises . . . 36

4 Data Visualization **37**
4.1 Introduction to ggplot2 . . . 37
4.2 Basic Plot Structure . . . 37
4.3 Common Plot Types . . . 37
4.3.1 Scatter Plots . . . 38
4.3.2 Line Plots . . . 38
4.3.3 Bar Plots . . . 39
4.3.4 Histograms and Density Plots . . . 40
4.3.5 Box Plots . . . 40
4.4 Customizing Plots . . . 41
4.4.1 Labels and Titles . . . 41
4.4.2 Scales and Axes . . . 41
4.4.3 Themes . . . 42
4.5 Faceting . . . 43
4.6 Advanced Visualizations . . . 43
4.6.1 Heatmaps . . . 43
4.6.2 Time Series Plots . . . 44
4.7 Practice Exercises . . . 44

5 Hypothesis Testing **45**
5.1 Introduction to Statistical Inference . . . 45
5.2 Hypothesis Testing Framework . . . 45
5.2.1 Null and Alternative Hypotheses . . . 45
5.2.2 Type I and Type II Errors . . . 45
5.3 One-Sample Tests . . . 46
5.3.1 One-Sample z-Test . . . 46
5.3.2 One-Sample t-Test . . . 47
5.4 Two-Sample Tests . . . 47
5.4.1 Independent Samples t-Test . . . 48
5.4.2 Paired Samples t-Test . . . 49
5.5 Analysis of Variance (ANOVA) . . . 49
5.5.1 One-Way ANOVA . . . 50
5.5.2 Two-Way ANOVA . . . 51
5.6 Assumption Checking . . . 51
5.6.1 Normality Tests . . . 51
5.6.2 Equal Variance Tests . . . 52
5.7 Effect Size Measures . . . 52
5.8 Power Analysis . . . 53
5.9 Practice Exercises . . . 53

6 Non-Parametric Tests **55**
6.1 Introduction to Non-Parametric Methods . . . 55
6.2 One-Sample Tests . . . 55

6.2.1 Sign Test . . . 55
6.2.2 Wilcoxon Signed-Rank Test . . . 56
6.3 Two-Sample Tests . . . 56
6.3.1 Mann-Whitney U Test (Wilcoxon Rank-Sum) . . . 57
6.3.2 Kolmogorov-Smirnov Test . . . 57
6.4 Multiple Group Tests . . . 58
6.4.1 Kruskal-Wallis Test . . . 58
6.4.2 Friedman Test . . . 58
6.5 Correlation Tests . . . 59
6.5.1 Spearman's Rank Correlation . . . 59
6.5.2 Kendall's Tau . . . 60
6.6 Chi-Square Tests . . . 60
6.6.1 Chi-Square Goodness of Fit . . . 60
6.6.2 Chi-Square Test of Independence . . . 61
6.7 Practice Exercises . . . 61

7 Repeated Measures ANOVA 62
7.1 Introduction . . . 62
7.2 Assumptions . . . 62
7.3 One-Way Repeated Measures ANOVA . . . 63
7.4 Two-Way Repeated Measures ANOVA . . . 64
7.5 Assumption Checking . . . 64
7.6 Post-Hoc Tests . . . 65
7.7 Effect Size . . . 65
7.8 Visualization . . . 66
7.9 Linear Mixed Effects Models . . . 66
7.10 Practice Exercises . . . 67

8 Regression Analysis 68
8.1 Simple Linear Regression . . . 68
8.2 Multiple Linear Regression . . . 69
8.3 Model Assessment . . . 70
8.4 Assumption Checking . . . 70
8.5 Logistic Regression . . . 71
8.6 Model Selection . . . 73
8.7 Practice Exercises . . . 73

9 Time Series Analysis and Forecasting 74
9.1 Introduction to Time Series . . . 74
9.2 Time Series Objects in R . . . 74
9.3 Decomposition . . . 75
9.4 Stationarity . . . 75
9.5 Autocorrelation . . . 76
9.6 ARIMA Models . . . 77
9.7 Exponential Smoothing . . . 78
9.8 Prophet . . . 78
9.9 Model Evaluation . . . 79
9.10 Practice Exercises . . . 79

10 Capstone Project **80**
10.1 Project Overview . . . 80
10.2 Project Timeline . . . 80
10.3 Project Structure . . . 80
10.3.1 1. Introduction . . . 80
10.3.2 2. Literature Review . . . 81
10.3.3 3. Methodology . . . 81
10.3.4 4. Results . . . 81
10.3.5 5. Discussion . . . 81
10.3.6 6. Conclusion . . . 81
10.4 Sample Project Topics . . . 82
10.5 Evaluation Criteria . . . 82
10.6 Report Guidelines . . . 83
10.7 Submission Requirements . . . 83
10.8 Practice Exercises . . . 84

A R Package Reference **85**

B Statistical Tables **85**
B.1 Critical Values for t-Distribution . . . 85

C Common R Errors and Solutions **85**

Preface

About These Notes

These comprehensive course notes have been prepared for students enrolled in the Data Analytics program at the Center for Data Analytics and Modeling (CDAM), Chuka University. The material covers foundational concepts essential for modern data analysis, combining statistical theory with practical R programming implementation.

Learning Approach: Each chapter combines theoretical foundations with hands-on R code examples. Students are encouraged to work through all code examples and complete the practice exercises at the end of each section.

Prerequisites: Basic mathematical knowledge (algebra, calculus fundamentals) and familiarity with computer operations. No prior programming experience required.

Software Requirements: R (version 4.0 or higher) and RStudio. All packages used in these notes are freely available from CRAN.

Contact: For questions and support, contact `lumumbavictor172@gmail.com` or visit CDAM offices at Chuka University.

1 Basic R Programming Skills

1.1 Introduction to R

R is a powerful programming language and environment specifically designed for statistical computing and graphics. Developed by Ross Ihaka and Robert Gentleman at the University of Auckland in 1993, R has become one of the most popular tools for data analysis worldwide.

Why Learn R?

- **Free and Open Source:** No licensing costs
- **Extensive Packages:** Over 18,000 packages on CRAN
- **Strong Community:** Active user support and documentation
- **Reproducible Research:** RMarkdown for dynamic reports
- **Industry Standard:** Widely used in academia and industry

1.2 RStudio Interface

RStudio is an integrated development environment (IDE) that makes working with R more efficient and user-friendly.

1.2.1 Main Panes

1. **Source Editor:** Write and edit R scripts
2. **Console:** Execute commands and view output
3. **Environment/History:** View objects and command history
4. **Files/Plots/Packages/Help:** Manage files and view visualizations

Your First R Code

```r
# This is a comment - R ignores this line

# Print a message
print("Hello, CDAM Students!")

# Simple calculation
2 + 2

# Assign a value to a variable
x <- 5
y <- 10
result <- x + y
print(result)
```

1.3 Data Types and Structures

1.3.1 Basic Data Types

R has five basic atomic data types:

Type	Description	Example
`numeric`	Decimal numbers	`3.14, -2.5`
`integer`	Whole numbers	`5L, 100L`
`character`	Text strings	`"hello", "CDAM"`
`logical`	Boolean values	`TRUE, FALSE`
`complex`	Complex numbers	`1+2i`

Table 1: Basic R Data Types

Checking Data Types

```r
# Check type of object
class(5)            # "numeric"
class(5L)           # "integer"
class("text")       # "character"
class(TRUE)         # "logical"

# Check structure
str(5)
typeof(5)

# Type conversion
as.numeric("10")    # 10
as.character(10)    # "10"
as.logical(1)       # TRUE
```

1.3.2 Data Structures

Vector Operations

A **vector** is the most basic data structure in R. All elements must be of the same type.

$$\mathbf{v} = (v_1, v_2, \ldots, v_n)$$

Vector operations are performed element-wise:

$$\mathbf{a} + \mathbf{b} = (a_1 + b_1, a_2 + b_2, \ldots, a_n + b_n)$$

Working with Vectors

```r
# Create vectors
v1 <- c(1, 2, 3, 4, 5)
v2 <- c(10, 20, 30, 40, 50)

# Vector operations
v1 + v2           # Element-wise addition
v1 * v2           # Element-wise multiplication
mean(v1)          # Mean: 3
sum(v1)           # Sum: 15
length(v1)        # Length: 5

# Subsetting
v1[1]             # First element
v1[1:3]           # First three elements
v1[v1 > 2]        # Elements greater than 2

# Create sequences
seq(1, 10, by=2)      # 1, 3, 5, 7, 9
1:10                  # 1, 2, 3, ..., 10
rep(1, times=5)       # 1, 1, 1, 1, 1
```

1.3.3 Matrices and Arrays

Matrices in R

```r
# Create matrix (column-wise by default)
m <- matrix(1:12, nrow=3, ncol=4)
print(m)

# Create matrix (row-wise)
m2 <- matrix(1:12, nrow=3, byrow=TRUE)

# Matrix operations
t(m)                # Transpose
m %*% t(m)          # Matrix multiplication
diag(m)             # Diagonal elements
det(m %*% t(m))     # Determinant

# Subsetting matrices
m[1, ]              # First row
m[, 2]              # Second column
m[1:2, 1:2]         # Submatrix
```

1.3.4 Data Frames

Data frames are the most important data structure for data analysis. They can contain different data types in different columns.

Working with Data Frames

```r
# Create data frame
df <- data.frame(
    id = 1:5,
    name = c("Alice", "Bob", "Charlie", "Diana", "Eve"),
    age = c(25, 30, 35, 28, 32),
    salary = c(50000, 60000, 75000, 55000, 65000),
    department = c("IT", "HR", "IT", "HR", "Finance")
)

# View data
head(df)            # First 6 rows
tail(df)            # Last 6 rows
str(df)             # Structure
summary(df)         # Summary statistics

# Access columns
df$name             # Using $
df[["name"]]        # Using [[
df[, "name"]        # Using [,]

# Access rows
df[1, ]             # First row
df[1:3, ]           # First three rows

# Add new column
df$bonus <- df$salary * 0.1
```

1.3.5 Lists

Working with Lists

```r
# Create list (can contain different types)
my_list <- list(
    numbers = 1:10,
    matrix = matrix(1:4, nrow=2),
    string = "Hello",
    nested = list(a=1, b=2)
)

# Access list elements
my_list[[1]]        # First element
my_list$numbers     # Named element
my_list$nested$a    # Nested element

# List operations
length(my_list)     # Number of elements
names(my_list)      # Element names
```

1.4 Control Structures

1.4.1 Conditional Statements

If-Else Statements

```r
# Simple if statement
x <- 10
if (x > 5) {
    print("x is greater than 5")
}

# If-else statement
if (x > 10) {
    print("x is greater than 10")
} else {
    print("x is 10 or less")
}

# If-else if-else
if (x > 15) {
    print("Large")
} else if (x > 5) {
    print("Medium")
} else {
    print("Small")
}

# Vectorized if (ifelse)
scores <- c(85, 92, 78, 65, 95)
grade <- ifelse(scores >= 80, "Pass", "Fail")
```

1.4.2 Loops

For and While Loops

```r
# For loop
for (i in 1:5) {
    print(paste("Iteration:", i))
}

# For loop with index
values <- c(10, 20, 30, 40, 50)
for (i in 1:length(values)) {
    print(values[i] * 2)
}

# While loop
count <- 1
while (count <= 5) {
    print(count)
    count <- count + 1
}

# Break and next
for (i in 1:10) {
    if (i == 3) next       # Skip iteration
    if (i == 7) break      # Exit loop
    print(i)
}
```

1.5 Functions

Function Definition

A function in R is defined as:

$$f(x_1, x_2, \ldots, x_n) = \text{expression}$$

General syntax:

```r
function_name <- function(arg1, arg2, ...) {
    # Function body
    result <- computation
    return(result)
}
```

Creating Functions

```r
# Simple function
add_numbers <- function(a, b) {
    return(a + b)
}
add_numbers(5, 10)    # 15

# Function with default arguments
greet <- function(name, greeting = "Hello") {
    paste(greeting, name)
}
greet("Alice")               # "Hello Alice"
greet("Bob", "Good morning") # "Good morning Bob"

# Function with multiple outputs
stats_summary <- function(x) {
    list(
        mean = mean(x),
        sd = sd(x),
        min = min(x),
        max = max(x)
    )
}
stats_summary(1:10)

# Anonymous function
sapply(1:5, function(x) x^2)
```

1.6 Apply Family Functions

Apply Functions

```r
# apply() - for matrices/arrays
m <- matrix(1:12, nrow=3)
apply(m, 1, mean)     # Row means
apply(m, 2, mean)     # Column means

# lapply() - returns list
list_data <- list(1:5, 6:10, 11:15)
lapply(list_data, mean)

# sapply() - returns vector/matrix
sapply(list_data, mean)

# tapply() - grouped operations
data <- c(10, 20, 30, 40, 50, 60)
group <- c("A", "A", "A", "B", "B", "B")
tapply(data, group, mean)

# mapply() - multivariate apply
mapply(function(x, y) x + y, 1:5, 6:10)
```

1.7 Error Handling and Debugging

Error Handling

```r
# try() - continue on error
result <- try(10/0, silent=TRUE)

# tryCatch() - handle specific errors
tryCatch({
    result <- 10/0
}, error = function(e) {
    print(paste("Error:", e$message))
}, warning = function(w) {
    print(paste("Warning:", w$message))
}, finally = {
    print("Execution complete")
})

# stopifnot() - assert conditions
x <- 5
stopifnot(x > 0, x < 10)

# Debugging tools
traceback()     # Show error stack
debug(function_name)    # Enter debug mode
browser()       # Insert breakpoint
```

1.8 Practice Exercises

1. Create a vector of numbers from 1 to 100. Calculate the sum, mean, and standard deviation.
2. Create a data frame with 5 columns: ID, Name, Age, Salary, Department. Add at least 10 rows of data.
3. Write a function that calculates the factorial of a number using both recursion and iteration.
4. Use apply functions to calculate column means for the built-in `mtcars` dataset.
5. Create a loop that prints all prime numbers between 1 and 50.

2 Data Importation and Cleaning

2.1 Overview

Data importation and cleaning typically consume 60-80% of a data analyst's time. Clean, well-structured data is essential for accurate analysis and reliable results.

The 80/20 Rule

In data science, approximately 80% of time is spent on data preparation (importing, cleaning, transforming), while only 20% is spent on actual analysis and modeling.

2.2 Importing Data from Various Sources

2.2.1 CSV and Text Files

Reading CSV Files

```r
# Base R
data <- read.csv("filename.csv",
                 header = TRUE,
                 stringsAsFactors = FALSE,
                 na.strings = c("", "NA", "N/A"))

# readr package (faster)
library(readr)
data <- read_csv("filename.csv",
                 col_types = cols(
                     id = col_integer(),
                     name = col_character(),
                     value = col_double()
                 ))

# Read tab-delimited
data <- read_tsv("filename.tsv")

# Read fixed-width
data <- read_fwf("filename.txt",
                 fwf_widths(c(10, 15, 8)))
```

2.2.2 Excel Files

Reading Excel Files

```r
library(readxl)

# Read specific sheet
data <- read_excel("filename.xlsx",
                   sheet = 1)

# Read by sheet name
data <- read_excel("filename.xlsx",
                   sheet = "Data")

# Read specific range
data <- read_excel("filename.xlsx",
                   range = "A1:D100")

# Get sheet names
excel_sheets("filename.xlsx")

# Multiple sheets
all_sheets <- excel_sheets("filename.xlsx")
data_list <- lapply(all_sheets,
                    function(x)
                        read_excel("filename.xlsx", sheet = x
    ))
```

2.2.3 Database Connections

Database Connections

```r
library(DBI)
library(RMySQL)     # or RPostgreSQL, RSQLite

# Connect to database
con <- dbConnect(MySQL(),
                 dbname = "database_name",
                 host = "localhost",
                 user = "username",
                 password = "password")

# List tables
dbListTables(con)

# Query data
data <- dbGetQuery(con,
                   "SELECT * FROM table_name WHERE condition"
  )

# Write to database
dbWriteTable(con, "new_table", data)

# Close connection
dbDisconnect(con)
```

2.2.4 Web Data and APIs

Web Scraping and APIs

```r
library(httr)
library(jsonlite)
library(rvest)

# GET request
response <- GET("https://api.example.com/data")
content <- content(response, "parsed")

# POST request with authentication
response <- POST("https://api.example.com/data",
                 body = list(key = "value"),
                 add_headers(Authorization = "Bearer token"))

# Parse JSON
data <- fromJSON("https://api.example.com/data.json")

# Web scraping
library(rvest)
page <- read_html("https://example.com")
tables <- html_nodes(page, "table")
data <- html_table(tables[[1]])
```

2.3 Data Quality Assessment

2.3.1 Identifying Data Quality Issues

Issue Type	Description	Detection Method
Missing Values	NA, blank cells	`sum(is.na(data))`
Duplicates	Repeated records	`duplicated(data)`
Outliers	Extreme values	Boxplots, Z-scores
Inconsistencies	Different formats	Pattern matching
Invalid Values	Out of range	Range checks
Type Errors	Wrong data types	`str(data)`

Table 2: Common Data Quality Issues

Data Quality Checks

```r
# Missing values
sum(is.na(data))
colSums(is.na(data))
mean(is.na(data)) * 100  # Percentage

# Duplicates
sum(duplicated(data))
data[duplicated(data), ]

# Unique values
sapply(data, function(x) length(unique(x)))

# Data types
str(data)
sapply(data, class)

# Basic statistics
summary(data)
describe(data)  # from psych package
```

2.4 Handling Missing Data

2.4.1 Types of Missing Data

Missing Data Mechanisms

1. **MCAR (Missing Completely At Random):** Missingness unrelated to any observed or unobserved data

$$P(M|X,Y) = P(M)$$

2. **MAR (Missing At Random):** Missingness related to observed data

$$P(M|X,Y) = P(M|X)$$

3. **MNAR (Missing Not At Random):** Missingness related to unobserved data

$$P(M|X,Y) \neq P(M|X)$$

Missing Data Strategies

```r
library(mice)
library(VIM)

# Visualize missing patterns
aggr(data, numbers=TRUE, sortVars=TRUE)
matrixplot(data)

# Remove missing (listwise deletion)
data_complete <- na.omit(data)

# Mean/Median imputation
data$age[is.na(data$age)] <-
    mean(data$age, na.rm=TRUE)

# Multiple imputation
imputed <- mice(data, m=5, method='pmm',
                maxit=50, seed=123)
data_complete <- complete(imputed)

# KNN imputation
library(VIM)
data_imputed <- kNN(data)

# Forward/backward fill (time series)
library(zoo)
data$value <- na.locf(data$value)  # Last observation
```

2.5 Data Cleaning Techniques

2.5.1 String Cleaning

String Manipulation

```r
library(stringr)

# Remove whitespace
str_trim("  hello  ")           # "hello"

# Change case
str_to_upper("hello")           # "HELLO"
str_to_lower("HELLO")           # "hello"
str_to_title("hello world")     # "Hello World"

# Replace text
str_replace("hello", "l", "L")  # "heLlo"
str_replace_all("hello", "l", "L")  # "heLLo"

# Extract patterns
str_extract("abc123", "\\d+")   # "123"
str_match("abc123", "(\\w+)(\\d+)")

# Split strings
str_split("a,b,c", ",")

# Detect patterns
str_detect("hello", "ell")      # TRUE
str_starts("hello", "h")        # TRUE
str_ends("hello", "o")          # TRUE
```

2.5.2 Date and Time Cleaning

Date-Time Operations

```r
library(lubridate)

# Parse dates
ymd("2024-01-15")
dmy("15/01/2024")
mdy("01/15/2024")
ymd_hms("2024-01-15 10:30:00")

# Extract components
date <- ymd("2024-01-15")
year(date)       # 2024
month(date)      # 1
day(date)        # 15
wday(date)       # Day of week

# Date arithmetic
date + days(7)
date + months(1)
date2 - date1    # Time difference

# Format dates
format(date, "%B %d, %Y")  # "January 15, 2024"
```

2.5.3 Outlier Detection and Treatment

Outlier Detection Methods

Z-Score Method:

$$z = \frac{x - \mu}{\sigma}$$

Values with $|z| > 3$ are typically considered outliers.

IQR Method:

$$\text{Lower Bound} = Q_1 - 1.5 \times \text{IQR}$$

$$\text{Upper Bound} = Q_3 + 1.5 \times \text{IQR}$$

where $\text{IQR} = Q_3 - Q_1$

Outlier Detection

```r
# Z-score method
z_scores <- abs(scale(data$value))
outliers_z <- which(z_scores > 3)

# IQR method
Q1 <- quantile(data$value, 0.25)
Q3 <- quantile(data$value, 0.75)
IQR <- Q3 - Q1
lower <- Q1 - 1.5 * IQR
upper <- Q3 + 1.5 * IQR
outliers_iqr <- which(data$value < lower |
                      data$value > upper)

# Visualize
boxplot(data$value, main="Boxplot")
hist(data$value, main="Histogram")

# Treatment options
# 1. Remove
data_clean <- data[-outliers_iqr, ]

# 2. Cap/Winsorize
library(DMwR)
data$value <- winsorize(data$value, limits=c(0.05, 0.95))

# 3. Transform
data$log_value <- log(data$value)
```

2.6 Data Validation

Data Validation Rules

```r
library(validate)
library(pointblank)

# Define validation rules
rules <- validator(
    age > 0 & age < 120,
    salary >= 0,
    !is.na(id),
    department %in% c("IT", "HR", "Finance")
)

# Apply validation
results <- confront(data, rules)
summary(results)

# pointblank for comprehensive validation
library(pointblank)
agent <- create_agent(tbl = data) %>%
    col_vals_gt(columns = age, value = 0) %>%
    col_vals_lt(columns = age, value = 120) %>%
    col_vals_not_null(columns = id) %>%
    interrogate()
agent %>% get_agent_report()
```

2.7 Data Export

Exporting Data

```r
# CSV
write.csv(data, "output.csv", row.names=FALSE)
write_csv(data, "output.csv")  # readr

# Excel
library(writexl)
write_xlsx(data, "output.xlsx")

# RDS (R native)
saveRDS(data, "output.rds")
data_loaded <- readRDS("output.rds")

# Multiple sheets
library(openxlsx)
wb <- createWorkbook()
addWorksheet(wb, "Sheet1")
addWorksheet(wb, "Sheet2")
writeData(wb, "Sheet1", data1)
writeData(wb, "Sheet2", data2)
saveWorkbook(wb, "output.xlsx")
```

2.8 Practice Exercises

1. Import a CSV file with missing values. Calculate the percentage of missing data per column and implement an appropriate imputation strategy.
2. Clean a dataset with inconsistent date formats. Convert all dates to a standard format and extract year, month, and day components.
3. Identify and handle outliers in a numerical column using both Z-score and IQR methods. Compare the results.
4. Create validation rules for a dataset and generate a data quality report.
5. Import data from multiple Excel sheets and combine them into a single data frame.

3 Data Manipulation and Wrangling

3.1 Introduction to dplyr

The `dplyr` package provides a consistent grammar for data manipulation. It is part of the tidyverse collection of packages.

The Five Main Verbs

1. `filter()` - Select rows
2. `select()` - Select columns
3. `mutate()` - Create/modify columns
4. `summarise()` - Aggregate data
5. `arrange()` - Sort data

3.2 Filtering Rows

Filter Operations

```r
library(dplyr)
library(nycflights13)

# Basic filter
flights_filtered <- filter(flights,
                           dest == "IAH")

# Multiple conditions (AND)
flights_filtered <- filter(flights,
                           dest == "IAH",
                           month == 1)

# OR conditions
flights_filtered <- filter(flights,
                           dest == "IAH" |
                           dest == "BOS")

# Using %in%
flights_filtered <- filter(flights,
                           dest %in% c("IAH", "BOS", "ORD"))

# Complex conditions
flights_filtered <- filter(flights,
                           air_time > 300,
                           distance > 1500)

# Filter with between
flights_filtered <- filter(flights,
                           between(month, 1, 3))
```

3.3 Selecting Columns

Select Operations

```
# Select specific columns
flights_selected <- select(flights,
                           year, month, day)

# Select range of columns
flights_selected <- select(flights,
                           year:day)

# Exclude columns
flights_selected <- select(flights,
                           -(year:day))

# Select with helpers
flights_selected <- select(flights,
                           starts_with("dep"))
flights_selected <- select(flights,
                           ends_with("time"))
flights_selected <- select(flights,
                           contains("time"))

# Rename while selecting
flights_selected <- select(flights,
                           year = year,
                           month = month,
                           dep_delay_min = dep_delay)

# Reorder columns
flights_selected <- select(flights,
                           dep_time, arr_time,
                           everything())
```

3.4 Creating and Modifying Columns

Mutate Operations

```r
# Create new column
flights_mutated <- mutate(flights,
                          speed = distance / air_time * 60)

# Multiple new columns
flights_mutated <- mutate(flights,
                          speed = distance / air_time * 60,
                          delay_total = dep_delay + arr_delay,
                          date = paste(year, month, day, sep="-"))

# Conditional mutation
flights_mutated <- mutate(flights,
                          delay_category = case_when(
                              dep_delay < 0 ~ "Early",
                              dep_delay <= 15 ~ "On Time",
                              dep_delay <= 60 ~ "Delayed",
                              TRUE ~ "Highly Delayed"
                          ))

# Modify existing column
flights_mutated <- mutate(flights,
                          dep_delay = dep_delay / 60)  # Convert to hours

# Remove column
flights_mutated <- select(flights, -dep_delay)
```

3.5 Summarizing Data

Common Summary Statistics

- **Mean:** $\bar{x} = \frac{1}{n}\sum_{i=1}^{n} x_i$
- **Median:** Middle value when sorted
- **Variance:** $s^2 = \frac{1}{n-1}\sum_{i=1}^{n}(x_i - \bar{x})^2$
- **Standard Deviation:** $s = \sqrt{s^2}$
- **Quantiles:** Q_p where $P(X \leq Q_p) = p$

Summarise Operations

```r
# Single summary
flights_summary <- summarise(flights,
                             avg_delay = mean(dep_delay, na.rm=TRUE))

# Multiple summaries
flights_summary <- summarise(flights,
                             avg_delay = mean(dep_delay, na.rm=TRUE),
                             sd_delay = sd(dep_delay, na.rm=TRUE),
                             min_delay = min(dep_delay, na.rm=TRUE),
                             max_delay = max(dep_delay, na.rm=TRUE),
                             count = n(),
                             missing = sum(is.na(dep_delay)))

# With quantiles
flights_summary <- summarise(flights,
                             q25 = quantile(dep_delay, 0.25, na.rm=TRUE),
                             q50 = quantile(dep_delay, 0.50, na.rm=TRUE),
                             q75 = quantile(dep_delay, 0.75, na.rm=TRUE))
```

3.6 Grouped Operations

Group By Operations

```r
# Group and summarise
flights_grouped <- flights %>%
    group_by(dest) %>%
    summarise(
        avg_delay = mean(dep_delay, na.rm=TRUE),
        count = n(),
        .groups = 'drop'
    )

# Multiple grouping variables
flights_grouped <- flights %>%
    group_by(carrier, dest) %>%
    summarise(
        avg_delay = mean(dep_delay, na.rm=TRUE),
        count = n(),
        .groups = 'drop'
    )

# Group and mutate (add group statistics)
flights_mutated <- flights %>%
    group_by(dest) %>%
    mutate(
        dest_avg_delay = mean(dep_delay, na.rm=TRUE),
        delay_from_avg = dep_delay - dest_avg_delay
    ) %>%
    ungroup()

# Filter after grouping
top_destinations <- flights %>%
    group_by(dest) %>%
    summarise(count = n()) %>%
    arrange(desc(count)) %>%
    head(10)
```

3.7 The Pipe Operator

The Pipe Operator

The pipe operator `%>%` passes the left-hand side as the first argument to the right-hand side function:

$$x\% > \%f(y) \quad \text{is equivalent to} \quad f(x, y)$$

This enables readable, left-to-right code flow.

Pipe Examples

```r
# Without pipe
filtered <- filter(flights, dest == "IAH")
selected <- select(filtered, year, month, dep_delay)
summarized <- summarise(selected, avg = mean(dep_delay))

# With pipe (cleaner)
result <- flights %>%
    filter(dest == "IAH") %>%
    select(year, month, dep_delay) %>%
    summarise(avg_delay = mean(dep_delay, na.rm=TRUE))

# Complex pipeline
analysis <- flights %>%
    filter(!is.na(dep_delay)) %>%
    group_by(carrier) %>%
    summarise(
        avg_delay = mean(dep_delay),
        count = n()
    ) %>%
    filter(count > 100) %>%
    arrange(desc(avg_delay)) %>%
    head(10)
```

3.8 Joining Data Frames

3.8.1 Types of Joins

Join Type	Description	R Function
Inner Join	Keep matching rows only	`inner_join()`
Left Join	Keep all left, match right	`left_join()`
Right Join	Keep all right, match left	`right_join()`
Full Join	Keep all rows from both	`full_join()`
Anti Join	Keep left without match	`anti_join()`
Semi Join	Keep left with match	`semi_join()`

Table 3: Join Types in dplyr

Join Operations

```r
# Create sample data
df1 <- data.frame(id = 1:5,
                  name = c("A", "B", "C", "D", "E"))
df2 <- data.frame(id = 3:7,
                  value = c(30, 40, 50, 60, 70))

# Inner join (matching only)
inner_join(df1, df2, by = "id")

# Left join (all from df1)
left_join(df1, df2, by = "id")

# Right join (all from df2)
right_join(df1, df2, by = "id")

# Full join (all from both)
full_join(df1, df2, by = "id")

# Anti join (df1 without match in df2)
anti_join(df1, df2, by = "id")

# Semi join (df1 with match in df2)
semi_join(df1, df2, by = "id")

# Join with different column names
left_join(df1, df2, by = c("id" = "customer_id"))
```

3.9 Reshaping Data

Pivot Operations

```r
library(tidyr)

# Wide to long (pivot_longer)
data_wide <- data.frame(
    id = 1:3,
    score_2020 = c(80, 85, 90),
    score_2021 = c(82, 87, 92),
    score_2022 = c(85, 89, 95)
)

data_long <- pivot_longer(data_wide,
                          cols = starts_with("score"),
                          names_to = "year",
                          values_to = "score")

# Long to wide (pivot_wider)
data_wide_back <- pivot_wider(data_long,
                              names_from = "year",
                              values_from = "score")

# Separate columns
data <- data.frame(date = c("2024-01-15", "2024-02-20"))
data_separated <- separate(data, date,
                           into = c("year", "month", "day"),
                           sep = "-")

# Unite columns
data_united <- unite(data_separated, "date",
                     year:day, sep = "-")
```

3.10 String and Date Manipulation

Advanced String Operations

```r
library(stringr)

# String detection
str_detect("hello world", "world")  # TRUE

# String extraction
str_extract("abc123def", "\\d+")    # "123"
str_extract_all("abc123def456", "\\d+")

# String replacement
str_replace("hello", "l", "L")
str_replace_all("hello", "l", "L")

# String splitting
str_split("a,b,c,d", ",")

# String padding
str_pad("5", width = 3, pad = "0")  # "005"

# String trimming
str_trim("  hello  ")

# Word extraction
word("hello world", 1)      # "hello"
word("hello world", -1)     # "world"
```

3.11 Practice Exercises

1. Using the `flights` dataset, find the top 10 destinations by average departure delay.
2. Create a summary table showing average delay by carrier and month.
3. Join the `flights` dataset with the `planes` dataset and analyze delay patterns by aircraft type.
4. Reshape a dataset from wide to long format and back.
5. Create a data pipeline that filters, transforms, groups, and summarizes data in a single chain.

4 Data Visualization

4.1 Introduction to ggplot2

The `ggplot2` package implements the Grammar of Graphics, a systematic approach to creating visualizations.

Grammar of Graphics

A plot is built from layers:

$$\text{Plot} = \text{Data} + \text{Aesthetics} + \text{Geometries} + \text{Facets} + \text{Themes}$$

- **Data:** The dataset to visualize
- **Aesthetics:** Mappings between variables and visual properties
- **Geometries:** The type of marks (points, lines, bars)
- **Facets:** Subplots for different groups
- **Themes:** Non-data visual elements

4.2 Basic Plot Structure

ggplot2 Basics

```r
library(ggplot2)

# Basic structure
ggplot(data = mpg,
       aes(x = displ, y = hwy)) +
    geom_point()

# With additional aesthetics
ggplot(data = mpg,
       aes(x = displ, y = hwy,
           color = class,
           size = cyl)) +
    geom_point(alpha = 0.5)

# Save plot
ggsave("plot.png", width = 8, height = 6, dpi = 300)
```

4.3 Common Plot Types

4.3.1 Scatter Plots

Scatter Plots

```
# Basic scatter plot
ggplot(mpg, aes(x = displ, y = hwy)) +
    geom_point()

# With regression line
ggplot(mpg, aes(x = displ, y = hwy)) +
    geom_point() +
    geom_smooth(method = "lm", se = TRUE)

# With multiple groups
ggplot(mpg, aes(x = displ, y = hwy,
                color = class)) +
    geom_point() +
    geom_smooth(method = "lm", se = FALSE)

# With labels
ggplot(mpg, aes(x = displ, y = hwy)) +
    geom_point() +
    geom_text(aes(label = model),
              size = 3,
              vjust = -0.5)
```

4.3.2 Line Plots

Line Plots

```
# Basic line plot
ggplot(economics, aes(x = date, y = unemploy)) +
    geom_line()

# Multiple lines
ggplot(economics, aes(x = date)) +
    geom_line(aes(y = unemploy, color = "Unemployed")) +
    geom_line(aes(y = uempmed, color = "Long-term")) +
    labs(color = "Metric")

# With points
ggplot(economics, aes(x = date, y = unemploy)) +
    geom_line() +
    geom_point()
```

4.3.3 Bar Plots

Bar Plots

```r
# Count bar plot
ggplot(mpg, aes(x = class)) +
    geom_bar()

# Filled bar plot
ggplot(mpg, aes(x = class, fill = drv)) +
    geom_bar()

# Proportional bar plot
ggplot(mpg, aes(x = class, fill = drv)) +
    geom_bar(position = "fill")

# Bar plot with values
ggplot(mpg, aes(x = class)) +
    geom_bar() +
    geom_text(stat = "count",
              aes(label = ..count..),
              vjust = -0.5)

# Horizontal bar plot
ggplot(mpg, aes(x = class)) +
    geom_bar() +
    coord_flip()
```

4.3.4 Histograms and Density Plots

Distribution Plots

```
# Histogram
ggplot(mpg, aes(x = hwy)) +
    geom_histogram(binwidth = 2,
                   fill = "blue",
                   alpha = 0.5)

# With density
ggplot(mpg, aes(x = hwy)) +
    geom_histogram(aes(y = ..density..),
                   binwidth = 2,
                   alpha = 0.5) +
    geom_density(color = "red", linewidth = 1)

# Faceted histogram
ggplot(mpg, aes(x = hwy)) +
    geom_histogram(binwidth = 2) +
    facet_wrap(~class)

# Density plot
ggplot(mpg, aes(x = hwy, color = class)) +
    geom_density()
```

4.3.5 Box Plots

Box Plots

```
# Basic box plot
ggplot(mpg, aes(x = class, y = hwy)) +
    geom_boxplot()

# With notches (confidence interval for median)
ggplot(mpg, aes(x = class, y = hwy)) +
    geom_boxplot(notch = TRUE)

# With jittered points
ggplot(mpg, aes(x = class, y = hwy)) +
    geom_boxplot() +
    geom_jitter(width = 0.2, alpha = 0.5)

# Violin plot (density + box)
ggplot(mpg, aes(x = class, y = hwy)) +
    geom_violin() +
    geom_boxplot(width = 0.1)
```

4.4 Customizing Plots

4.4.1 Labels and Titles

Labels

```
ggplot(mpg, aes(x = displ, y = hwy)) +
    geom_point() +
    labs(
        title = "Engine Displacement vs Highway MPG",
        subtitle = "Data from EPA fuel economy database",
        caption = "Source: ggplot2 package",
        x = "Engine Displacement (L)",
        y = "Highway Miles Per Gallon",
        color = "Vehicle Class"
    )
```

4.4.2 Scales and Axes

Scales

```
# Color scales
ggplot(mpg, aes(x = displ, y = hwy, color = class)) +
    geom_point() +
    scale_color_brewer(palette = "Set1")

# Continuous color scale
ggplot(mpg, aes(x = displ, y = hwy, color = hwy)) +
    geom_point() +
    scale_color_gradient(low = "blue", high = "red")

# Axis limits
ggplot(mpg, aes(x = displ, y = hwy)) +
    geom_point() +
    xlim(1, 6) +
    ylim(10, 40)

# Log scale
ggplot(mpg, aes(x = displ, y = hwy)) +
    geom_point() +
    scale_y_log10()

# Custom breaks
ggplot(mpg, aes(x = displ, y = hwy)) +
    geom_point() +
    scale_x_continuous(breaks = seq(1, 7, by = 1))
```

4.4.3 Themes

Themes

```r
# Built-in themes
ggplot(mpg, aes(x = displ, y = hwy)) +
    geom_point() +
    theme_minimal()

ggplot(mpg, aes(x = displ, y = hwy)) +
    geom_point() +
    theme_bw()

ggplot(mpg, aes(x = displ, y = hwy)) +
    geom_point() +
    theme_classic()

# Custom theme
ggplot(mpg, aes(x = displ, y = hwy)) +
    geom_point() +
    theme(
        plot.title = element_text(size = 16, face = "bold"),
        axis.title = element_text(size = 12),
        axis.text = element_text(size = 10),
        panel.grid.major = element_line(color = "gray80"),
        panel.background = element_rect(fill = "white")
    )
```

4.5 Faceting

Facets

```
# Wrap (single variable)
ggplot(mpg, aes(x = displ, y = hwy)) +
    geom_point() +
    facet_wrap(~class)

# Wrap with multiple rows
ggplot(mpg, aes(x = displ, y = hwy)) +
    geom_point() +
    facet_wrap(~class, nrow = 2)

# Grid (two variables)
ggplot(mpg, aes(x = displ, y = hwy)) +
    geom_point() +
    facet_grid(drv ~ cyl)

# With free scales
ggplot(mpg, aes(x = displ, y = hwy)) +
    geom_point() +
    facet_wrap(~class, scales = "free")
```

4.6 Advanced Visualizations

4.6.1 Heatmaps

Heatmaps

```
# Create correlation matrix
cor_matrix <- cor(mtcars)

# Heatmap
library(reshape2)
cor_melt <- melt(cor_matrix)

ggplot(cor_melt, aes(x = Var1, y = Var2, fill = value)) +
    geom_tile() +
    geom_text(aes(label = round(value, 2)),
              color = "white") +
    scale_fill_gradient2(low = "blue",
                         mid = "white",
                         high = "red",
                         midpoint = 0) +
    theme_minimal() +
    theme(axis.text.x = element_text(angle = 45, hjust = 1))
```

4.6.2 Time Series Plots

Time Series

```r
library(lubridate)

# Basic time series
ggplot(economics, aes(x = date, y = unemploy)) +
    geom_line() +
    scale_x_date(date_breaks = "5 years",
                 date_labels = "%Y")

# With annotations
ggplot(economics, aes(x = date, y = unemploy)) +
    geom_line() +
    annotate("rect",
             xmin = as.Date("2008-01-01"),
             xmax = as.Date("2009-12-31"),
             ymin = -Inf, ymax = Inf,
             alpha = 0.2, fill = "red") +
    annotate("text",
             x = as.Date("2008-06-01"),
             y = 15000,
             label = "Financial Crisis")
```

4.7 Practice Exercises

1. Create a scatter plot with regression line showing the relationship between two variables of your choice.
2. Create a faceted histogram showing the distribution of a variable across different groups.
3. Design a dashboard-style visualization with multiple plot types combined.
4. Create a time series plot with custom annotations highlighting important events.
5. Design a custom theme for your organization's reports.

5 Hypothesis Testing

5.1 Introduction to Statistical Inference

Statistical inference allows us to make conclusions about population parameters based on sample data. Hypothesis testing is a formal procedure for investigating our ideas about the world.

Key Concepts

- **Population:** Entire group of interest
- **Sample:** Subset of population used for analysis
- **Parameter:** Numerical characteristic of population (e.g., μ, σ)
- **Statistic:** Numerical characteristic of sample (e.g., $\bar{x}$, s)
- **Sampling Distribution:** Distribution of a statistic over repeated samples

5.2 Hypothesis Testing Framework

5.2.1 Null and Alternative Hypotheses

Hypothesis Formulation

Null Hypothesis (H_0): Statement of no effect or no difference. Always contains equality.
Alternative Hypothesis (H_1 or H_a): Statement we want to prove. Contains inequality.

Examples:

- Two-tailed: $H_0 : \mu = \mu_0$ vs. $H_1 : \mu \neq \mu_0$
- One-tailed (right): $H_0 : \mu \leq \mu_0$ vs. $H_1 : \mu > \mu_0$
- One-tailed (left): $H_0 : \mu \geq \mu_0$ vs. $H_1 : \mu < \mu_0$

5.2.2 Type I and Type II Errors

	H_0 **True**	H_0 **False**
Reject H_0	Type I Error (α)	Correct (Power $= 1 - \beta$)
Fail to Reject H_0	Correct	Type II Error (β)

Table 4: Decision Matrix for Hypothesis Testing

Error Rates

Significance Level (α): Probability of Type I error (typically 0.05)
Power ($1 - \beta$): Probability of correctly rejecting false H_0 (typically 0.80)
Relationship: Decreasing α increases β (trade-off)

5.3 One-Sample Tests

5.3.1 One-Sample z-Test

z-Test Formula

When population standard deviation σ is known:

$$z = \frac{\bar{x} - \mu_0}{\sigma/\sqrt{n}}$$

where:

- $\bar{x}$ = sample mean
- μ_0 = hypothesized population mean
- σ = population standard deviation
- n = sample size

Assumptions:

- Random sampling
- Normal population or $n \geq 30$
- Known σ

z-Test in R

```r
# Using BSDA package
library(BSDA)

# One-sample z-test
z.test(x = sample_data,
       sigma.x = known_sd,
       mu = hypothesized_mean,
       alternative = "two.sided",
       conf.level = 0.95)
```

5.3.2 One-Sample t-Test

t-Test Formula

When population standard deviation is unknown:

$$t = \frac{\bar{x} - \mu_0}{s/\sqrt{n}}$$

with degrees of freedom $df = n - 1$
Assumptions:

- Random sampling
- Normal population (or $n \geq 30$)
- Unknown σ (estimated by s)

t-Test in R

```r
# One-sample t-test
t.test(x = sample_data,
       mu = hypothesized_mean,
       alternative = "two.sided",
       conf.level = 0.95)

# Example: Test if mean age is 30
ages <- c(25, 32, 28, 35, 30, 29, 33, 31, 27, 34)
t.test(ages, mu = 30)

# Output interpretation
# t = test statistic
# df = degrees of freedom
# p-value = probability of observing this result if H0 is
  true
# conf.int = 95% confidence interval for mean
```

5.4 Two-Sample Tests

5.4.1 Independent Samples t-Test

Independent t-Test Formula

Equal Variances (Student's t-test):

$$t = \frac{\bar{x}_1 - \bar{x}_2}{s_p\sqrt{\frac{1}{n_1} + \frac{1}{n_2}}}$$

where pooled standard deviation:

$$s_p = \sqrt{\frac{(n_1 - 1)s_1^2 + (n_2 - 1)s_2^2}{n_1 + n_2 - 2}}$$

Unequal Variances (Welch's t-test):

$$t = \frac{\bar{x}_1 - \bar{x}_2}{\sqrt{\frac{s_1^2}{n_1} + \frac{s_2^2}{n_2}}}$$

with Welch-Satterthwaite degrees of freedom.

Independent t-Test in R

```r
# Two-sample t-test
# Default is Welch's test (var.equal = FALSE)
t.test(outcome ~ group,
       data = dataset,
       alternative = "two.sided",
       conf.level = 0.95)

# Student's t-test (assume equal variance)
t.test(outcome ~ group,
       data = dataset,
       var.equal = TRUE)

# Test for equal variance first
var.test(outcome ~ group, data = dataset)

# Example: Compare test scores between two classes
class_a <- c(85, 88, 90, 87, 89, 92, 86, 88)
class_b <- c(78, 82, 85, 80, 83, 79, 81, 84)
t.test(class_a, class_b, var.equal = FALSE)
```

5.4.2 Paired Samples t-Test

Paired t-Test Formula

For paired/matched observations:

$$t = \frac{\bar{d}}{s_d/\sqrt{n}}$$

where:

- $\bar{d}$ = mean of differences
- s_d = standard deviation of differences
- n = number of pairs

Degrees of Freedom: $df = n - 1$

Paired t-Test in R

```r
# Paired t-test
t.test(before, after,
       paired = TRUE,
       alternative = "two.sided")

# Example: Pre-test and post-test scores
before <- c(65, 70, 68, 72, 69, 71, 67, 73)
after <- c(75, 78, 76, 80, 77, 79, 74, 81)
t.test(before, after, paired = TRUE)

# Interpretation
# If p < 0.05: Significant difference between before and
   after
```

5.5 Analysis of Variance (ANOVA)

5.5.1 One-Way ANOVA

ANOVA Framework

Hypotheses:

- $H_0 : \mu_1 = \mu_2 = \cdots = \mu_k$ (all means equal)
- H_1 : At least one mean differs

F-Statistic:

$$F = \frac{\text{MS}_{\text{between}}}{\text{MS}_{\text{within}}} = \frac{\text{SS}_{\text{between}}/(k-1)}{\text{SS}_{\text{within}}/(N-k)}$$

where:

- k = number of groups
- N = total sample size
- SS = Sum of Squares
- MS = Mean Square

One-Way ANOVA in R

```r
# One-way ANOVA
model <- aov(outcome ~ group, data = dataset)
summary(model)

# Example: Compare yields across fertilizers
yield <- c(20, 22, 21, 23, 18, 19, 20, 25, 26, 24, 22, 23)
fertilizer <- factor(c("A", "A", "A", "A",
                       "B", "B", "B", "B",
                       "C", "C", "C", "C"))
data <- data.frame(yield, fertilizer)

model <- aov(yield ~ fertilizer, data = data)
summary(model)

# Post-hoc tests (if ANOVA significant)
TukeyHSD(model)

# Check assumptions
plot(model)  # Residual plots
shapiro.test(residuals(model))  # Normality
leveneTest(yield ~ fertilizer, data = data)  # Equal variance
```

5.5.2 Two-Way ANOVA

Two-Way ANOVA

```
# Two-way ANOVA with interaction
model <- aov(outcome ~ factor1 * factor2,
             data = dataset)
summary(model)

# Without interaction
model <- aov(outcome ~ factor1 + factor2,
             data = dataset)

# Example: Effect of fertilizer and irrigation
yield <- c(20, 22, 18, 19, 25, 26, 22, 23,
           21, 23, 19, 20, 26, 27, 23, 24)
fertilizer <- factor(rep(c("A", "B"), each = 8))
irrigation <- factor(rep(c("Low", "High"), times = 8))
data <- data.frame(yield, fertilizer, irrigation)

model <- aov(yield ~ fertilizer * irrigation, data = data)
summary(model)

# Interaction plot
interaction.plot(fertilizer, irrigation, yield)
```

5.6 Assumption Checking

5.6.1 Normality Tests

Normality Testing

```
# Shapiro-Wilk test (n < 50)
shapiro.test(data)

# Kolmogorov-Smirnov test (n >= 50)
ks.test(data, "pnorm",
        mean = mean(data),
        sd = sd(data))

# Visual checks
qqnorm(data)
qqline(data, col = "red")

hist(data, breaks = 10, freq = FALSE)
curve(dnorm(x, mean(data), sd(data)),
      add = TRUE, col = "red")
```

5.6.2 Equal Variance Tests

Variance Tests

```r
library(car)

# Levene's test (robust to non-normality)
leveneTest(outcome ~ group, data = dataset)

# Bartlett's test (requires normality)
bartlett.test(outcome ~ group, data = dataset)

# F-test (two groups only)
var.test(outcome ~ group, data = dataset)

# If variances unequal: use Welch's test
t.test(outcome ~ group, data = dataset,
       var.equal = FALSE)
```

5.7 Effect Size Measures

Effect Size Formulas

Cohen's d (t-tests):

$$d = \frac{\bar{x}_1 - \bar{x}_2}{s_{\text{pooled}}}$$

Interpretation: 0.2 = small, 0.5 = medium, 0.8 = large

Eta-squared (ANOVA):

$$\eta^2 = \frac{\text{SS}_{\text{between}}}{\text{SS}_{\text{total}}}$$

Pearson's r (correlation):

$$r = \frac{\sum(x_i - \bar{x})(y_i - \bar{y})}{\sqrt{\sum(x_i - \bar{x})^2 \sum(y_i - \bar{y})^2}}$$

Effect Size in R

```r
library(effsize)

# Cohen's d
cohen.d(outcome ~ group, data = dataset)

# Eta-squared
library(effectsize)
eta_squared(model)

# Interpretation guidelines
# Cohen's d: 0.2 (small), 0.5 (medium), 0.8 (large)
# Eta-squared: 0.01 (small), 0.06 (medium), 0.14 (large)
```

5.8 Power Analysis

Power Analysis

```r
library(pwr)

# Power for t-test
pwr.t.test(n = 30, d = 0.5,
           sig.level = 0.05,
           type = "two.sample",
           alternative = "two.sided")

# Calculate required sample size
pwr.t.test(d = 0.5, power = 0.80,
           sig.level = 0.05,
           type = "two.sample")

# Power for ANOVA
pwr.anova.test(k = 3, f = 0.25,
               sig.level = 0.05, power = 0.80)

# Power for correlation
pwr.r.test(r = 0.3, power = 0.80,
           sig.level = 0.05)
```

5.9 Practice Exercises

1. Conduct a one-sample t-test to determine if the mean of a sample differs from a specified value.
2. Perform an independent samples t-test to compare two groups. Check all assumptions.
3. Conduct a one-way ANOVA with post-hoc tests. Report effect sizes.

4. Calculate the required sample size for a study with 80% power to detect a medium effect.

5. Interpret the results of hypothesis tests in the context of a real-world problem.

6 Non-Parametric Tests

6.1 Introduction to Non-Parametric Methods

Non-parametric tests do not assume a specific distribution for the population. They are useful when:

- Data violates normality assumption
- Sample size is small
- Data is ordinal or ranked
- Outliers are present

When to Use Non-Parametric Tests

- Shapiro-Wilk test indicates non-normality (p ¡ 0.05)
- Small sample size (n ¡ 30) with unknown distribution
- Ordinal data (Likert scales, rankings)
- Presence of significant outliers

6.2 One-Sample Tests

6.2.1 Sign Test

Sign Test

Tests whether the median differs from a hypothesized value.
Procedure:

1. Calculate differences: $d_i = x_i - M_0$
2. Count positive and negative signs
3. Test statistic follows binomial distribution

$$H_0 : \text{Median} = M_0$$
$$H_1 : \text{Median} \neq M_0$$

Sign Test in R

```r
library(BSDA)

# Sign test
SIGN.test(x = data,
          md = hypothesized_median,
          alternative = "two.sided")

# Example
data <- c(12, 15, 10, 18, 14, 16, 11, 17)
SIGN.test(data, md = 15)
```

6.2.2 Wilcoxon Signed-Rank Test

Wilcoxon Signed-Rank Test

More powerful than sign test. Uses magnitude of differences.
Procedure:

1. Calculate differences: $d_i = x_i - M_0$
2. Rank absolute differences (ignore zeros)
3. Sum ranks for positive and negative differences
4. Test statistic $W = \min(W^+, W^-)$

Wilcoxon Signed-Rank in R

```r
# One-sample Wilcoxon test
wilcox.test(x = data,
            mu = hypothesized_median,
            alternative = "two.sided",
            exact = TRUE)

# Example
data <- c(12, 15, 10, 18, 14, 16, 11, 17)
wilcox.test(data, mu = 15)

# Paired Wilcoxon test
before <- c(10, 12, 11, 13, 14, 12, 11, 15)
after <- c(14, 15, 13, 16, 17, 14, 13, 18)
wilcox.test(before, after, paired = TRUE)
```

6.3 Two-Sample Tests

6.3.1 Mann-Whitney U Test (Wilcoxon Rank-Sum)

Mann-Whitney U Test

Non-parametric alternative to independent t-test.
Hypotheses:

- H_0 : Distributions are equal
- H_1 : Distributions differ (or one is stochastically larger)

Test Statistic:

$$U = n_1 n_2 + \frac{n_1(n_1+1)}{2} - R_1$$

where R_1 is the sum of ranks for group 1.

Mann-Whitney U in R

```r
# Wilcoxon rank-sum test
wilcox.test(outcome ~ group,
            data = dataset,
            alternative = "two.sided",
            exact = TRUE)

# Example
group_a <- c(10, 12, 11, 13, 14, 12, 11, 15)
group_b <- c(18, 20, 19, 22, 21, 19, 20, 23)
wilcox.test(group_a, group_b)

# With formula interface
wilcox.test(score ~ treatment, data = clinical_data)
```

6.3.2 Kolmogorov-Smirnov Test

K-S Test

```r
# Two-sample K-S test
ks.test(sample1, sample2)

# One-sample K-S test (against distribution)
ks.test(data, "pnorm",
        mean = mean(data),
        sd = sd(data))

# Example: Compare two distributions
group1 <- rnorm(50, mean = 10, sd = 2)
group2 <- rnorm(50, mean = 12, sd = 2)
ks.test(group1, group2)
```

6.4 Multiple Group Tests

6.4.1 Kruskal-Wallis Test

Kruskal-Wallis Test

Non-parametric alternative to one-way ANOVA.
Hypotheses:

- H_0 : All groups have same distribution
- H_1 : At least one group differs

Test Statistic:

$$H = \frac{12}{N(N+1)} \sum_{i=1}^{k} \frac{R_i^2}{n_i} - 3(N+1)$$

where R_i is sum of ranks for group i.

Kruskal-Wallis in R

```r
# Kruskal-Wallis test
kruskal.test(outcome ~ group, data = dataset)

# Example
yield <- c(20, 22, 21, 23, 18, 19, 20, 25, 26, 24, 22, 23)
fertilizer <- factor(c("A", "A", "A", "A",
                       "B", "B", "B", "B",
                       "C", "C", "C", "C"))
data <- data.frame(yield, fertilizer)
kruskal.test(yield ~ fertilizer, data = data)

# Post-hoc pairwise comparisons
library(dunn.test)
dunn.test(data$yield, data$fertilizer,
          method = "bonferroni")
```

6.4.2 Friedman Test

Friedman Test

Non-parametric alternative to repeated measures ANOVA.
Use when: Same subjects measured under multiple conditions.
Test Statistic:

$$\chi_r^2 = \frac{12}{nk(k+1)} \sum_{j=1}^{k} R_j^2 - 3n(k+1)$$

where R_j is sum of ranks for condition j.

Friedman Test in R

```
# Friedman test
friedman.test(outcome ~ condition | subject,
              data = dataset)

# Example: Repeated measures
score <- c(10, 12, 11, 14, 15, 13, 16, 17, 15)
condition <- factor(rep(c("A", "B", "C"), each = 3))
subject <- factor(rep(1:3, times = 3))
data <- data.frame(score, condition, subject)

friedman.test(score ~ condition | subject, data = data)

# Post-hoc
library(PMCMRplus)
frdAllPairsConoverTest(score ~ condition | subject,
                       data = data)
```

6.5 Correlation Tests

6.5.1 Spearman's Rank Correlation

Spearman's Rho

Non-parametric measure of monotonic association.

$$\rho = 1 - \frac{6\sum d_i^2}{n(n^2-1)}$$

where d_i is difference between ranks of x_i and y_i.
Range: $-1 \leq \rho \leq 1$

Spearman Correlation

```
# Spearman correlation
cor.test(x, y, method = "spearman")

# Example
x <- c(10, 20, 30, 40, 50)
y <- c(15, 25, 35, 45, 55)
cor.test(x, y, method = "spearman")

# Correlation matrix
cor(data, method = "spearman")
```

6.5.2 Kendall's Tau

Kendall's Tau

```r
# Kendall correlation
cor.test(x, y, method = "kendall")

# Comparison
cor.test(x, y, method = "pearson")   # Parametric
cor.test(x, y, method = "spearman")  # Non-parametric
cor.test(x, y, method = "kendall")   # Non-parametric
```

6.6 Chi-Square Tests

6.6.1 Chi-Square Goodness of Fit

Chi-Square Goodness of Fit

Tests if observed frequencies match expected frequencies.

$$\chi^2 = \sum_{i=1}^{k} \frac{(O_i - E_i)^2}{E_i}$$

Degrees of Freedom: $df = k - 1 - m$ (where m = estimated parameters)

Chi-Square Goodness of Fit

```r
# Chi-square goodness of fit
observed <- c(50, 30, 20)
expected <- c(0.5, 0.3, 0.2)
chisq.test(observed, p = expected)

# Example: Test if die is fair
observed <- c(95, 105, 98, 102, 97, 103)
expected <- rep(1/6, 6)
chisq.test(observed, p = expected)
```

6.6.2 Chi-Square Test of Independence

Chi-Square Independence Test

```r
# Create contingency table
table <- matrix(c(50, 30, 20, 40),
                nrow = 2,
                dimnames = list(
                    Gender = c("Male", "Female"),
                    Vote = c("Yes", "No")
                ))

# Chi-square test
chisq.test(table)

# With data frame
chisq.test(dataset$gender, dataset$vote)

# Fisher's exact test (for small samples)
fisher.test(table)

# Effect size (Cramer's V)
library(vcd)
assocstats(table)
```

6.7 Practice Exercises

1. Conduct a Wilcoxon signed-rank test for paired data that violates normality.
2. Perform a Mann-Whitney U test to compare two independent groups.
3. Use Kruskal-Wallis test for comparing more than two groups.
4. Calculate Spearman correlation for ordinal data.
5. Perform chi-square tests for categorical data analysis.

7 Repeated Measures ANOVA

7.1 Introduction

Repeated measures ANOVA is used when the same subjects are measured multiple times under different conditions or at different time points.

When to Use Repeated Measures ANOVA

- Same subjects measured at multiple time points
- Same subjects under different treatment conditions
- Matched or paired designs
- Longitudinal studies

7.2 Assumptions

Repeated Measures ANOVA Assumptions

1. **Normality:** Dependent variable normally distributed for each condition
2. **Sphericity:** Variances of differences between all pairs of conditions are equal
3. **Independence:** Subjects are independent of each other
4. **Continuous DV:** Dependent variable is continuous

Sphericity Test (Mauchly's):

$$H_0 : \text{Sphericity holds}$$

If violated, use Greenhouse-Geisser or Huynh-Feldt corrections.

7.3 One-Way Repeated Measures ANOVA

One-Way Repeated Measures Model

$$Y_{ij} = \mu + \alpha_i + \pi_j + \epsilon_{ij}$$

where:

- Y_{ij} = observation for subject j in condition i
- μ = grand mean
- α_i = effect of condition i
- π_j = effect of subject j
- ϵ_{ij} = random error

One-Way Repeated Measures in R

```r
library(rstatix)
library(afex)

# Data in long format
# subject | condition | score

# Using rstatix
anova_results <- data %>%
    anova_test(dv = score,
               wid = subject,
               within = condition)

# View results
anova_results

# Mauchly's test for sphericity
anova_results %>% get_anova_table()

# If sphericity violated, use corrected p-values
# (Greenhouse-Geisser or Huynh-Feldt)
```

7.4 Two-Way Repeated Measures ANOVA

Two-Way Repeated Measures

```r
# Two within-subjects factors
anova_results <- data %>%
    anova_test(dv = score,
               wid = subject,
               within = c(time, treatment))

# Mixed design (one within, one between)
anova_results <- data %>%
    anova_test(dv = score,
               wid = subject,
               within = time,
               between = group)

# View full table
get_anova_table(anova_results,
                correction = "auto")
```

7.5 Assumption Checking

Assumption Checks

```r
# Normality (by condition)
data %>%
    group_by(condition) %>%
    shapiro_test(score)

# Sphericity (Mauchly's test)
anova_results %>% get_anova_table()
# Look at Mauchly's test p-value

# If p < 0.05, sphericity violated
# Use Greenhouse-Geisser correction

# Outliers
data %>%
    group_by(subject, condition) %>%
    identify_outliers(score)

# QQ plot
data %>%
    ggqqplot(score, facet.by = "condition")
```

7.6 Post-Hoc Tests

Post-Hoc Comparisons

```
# Pairwise comparisons with Bonferroni correction
pairwise_results <- data %>%
    pairwise_t_test(score ~ condition,
                    paired = TRUE,
                    p.adjust.method = "bonferroni")

# With Holm correction (more powerful)
pairwise_results <- data %>%
    pairwise_t_test(score ~ condition,
                    paired = TRUE,
                    p.adjust.method = "holm")

# View results
pairwise_results

# Add significance stars
pairwise_results %>%
    add_significance()
```

7.7 Effect Size

Effect Size for Repeated Measures

```
# Generalized eta-squared
anova_results %>%
    get_anova_table() %>%
    select(Effect, GES)  # GES = Generalized Eta-Squared

# Partial eta-squared
anova_results %>%
    get_anova_table(correction = "none") %>%
    select(Effect, pes)

# Interpretation (GES)
# 0.01 = small, 0.06 = medium, 0.14 = large
```

7.8 Visualization

Visualization

```r
library(ggplot2)

# Mean plot with error bars
data %>%
    group_by(condition) %>%
    summarise(
        mean = mean(score),
        sd = sd(score),
        n = n(),
        se = sd / sqrt(n)
    ) %>%
    ggplot(aes(x = condition, y = mean)) +
    geom_point(size = 3) +
    geom_line(group = 1) +
    geom_errorbar(aes(ymin = mean - se,
                      ymax = mean + se),
                  width = 0.2) +
    theme_minimal()

# Individual trajectories
data %>%
    ggplot(aes(x = condition, y = score,
               group = subject)) +
    geom_line(alpha = 0.3) +
    geom_point() +
    stat_summary(fun = mean,
                 geom = "line",
                 color = "red",
                 size = 1.5) +
    theme_minimal()
```

7.9 Linear Mixed Effects Models

Mixed Effects Model

For more complex designs, use linear mixed models:

$$Y_{ij} = \beta_0 + \beta_1 X_{ij} + u_j + \epsilon_{ij}$$

where $u_j \sim N(0, \sigma_u^2)$ is the random effect for subject j.

Mixed Effects in R

```r
library(lme4)
library(lmerTest)

# Random intercept model
model <- lmer(score ~ condition + (1 | subject),
              data = data)

# Random intercept and slope
model <- lmer(score ~ condition +
              (1 + condition | subject),
              data = data)

# View results
summary(model)
anova(model)

# Check assumptions
plot(model)  # Residual plots
qqnorm(resid(model))
qqline(resid(model))
```

7.10 Practice Exercises

1. Conduct a one-way repeated measures ANOVA for a pre-post-follow-up design.
2. Perform assumption checks including sphericity test.
3. Run appropriate post-hoc tests with corrections.
4. Calculate and interpret effect sizes.
5. Create visualizations showing individual trajectories and group means.

8 Regression Analysis

8.1 Simple Linear Regression

Simple Linear Regression Model

$$Y_i = \beta_0 + \beta_1 X_i + \epsilon_i$$

where:

- Y_i = dependent variable
- X_i = independent variable
- β_0 = intercept
- β_1 = slope
- $\epsilon_i \sim N(0, \sigma^2)$ = error term

Least Squares Estimation:

$$\hat{\beta}_1 = \frac{\sum(x_i - \bar{x})(y_i - \bar{y})}{\sum(x_i - \bar{x})^2}$$

$$\hat{\beta}_0 = \bar{y} - \hat{\beta}_1 \bar{x}$$

Simple Linear Regression in R

```r
# Fit model
model <- lm(y ~ x, data = dataset)

# View results
summary(model)

# Coefficients
coef(model)

# Confidence intervals
confint(model, level = 0.95)

# Predictions
predictions <- predict(model,
                       newdata = new_dataset,
                       interval = "confidence")

# Example
model <- lm(mpg ~ wt, data = mtcars)
summary(model)
```

8.2 Multiple Linear Regression

Multiple Regression Model

$$Y_i = \beta_0 + \beta_1 X_{i1} + \beta_2 X_{i2} + \cdots + \beta_p X_{ip} + \epsilon_i$$

In matrix form:

$$\mathbf{Y} = \mathbf{X}\boldsymbol{\beta} + \boldsymbol{\epsilon}$$

OLS Estimator:

$$\hat{\boldsymbol{\beta}} = (\mathbf{X}^T\mathbf{X})^{-1}\mathbf{X}^T\mathbf{Y}$$

Multiple Regression in R

```r
# Multiple regression
model <- lm(y ~ x1 + x2 + x3, data = dataset)

# With interaction
model <- lm(y ~ x1 * x2, data = dataset)

# With polynomial
model <- lm(y ~ poly(x, 2), data = dataset)

# View results
summary(model)

# ANOVA table
anova(model)

# Example
model <- lm(mpg ~ wt + hp + cyl, data = mtcars)
summary(model)
```

8.3 Model Assessment

Regression Metrics

R-squared:

$$R^2 = 1 - \frac{SS_{res}}{SS_{tot}} = 1 - \frac{\sum(y_i - \hat{y}_i)^2}{\sum(y_i - \bar{y})^2}$$

Adjusted R-squared:

$$R^2_{adj} = 1 - \frac{(1 - R^2)(n - 1)}{n - p - 1}$$

Root Mean Square Error:

$$RMSE = \sqrt{\frac{1}{n}\sum_{i=1}^{n}(y_i - \hat{y}_i)^2}$$

Model Assessment in R

```r
# R-squared
summary(model)$r.squared
summary(model)$adj.r.squared

# RMSE
library(Metrics)
rmse(actual = dataset$y,
     predicted = predict(model))

# AIC and BIC
AIC(model)
BIC(model)

# F-statistic
summary(model)$fstatistic
```

8.4 Assumption Checking

Regression Assumptions

1. **Linearity:** Relationship between X and Y is linear
2. **Independence:** Observations are independent
3. **Homoscedasticity:** Constant variance of residuals
4. **Normality:** Residuals are normally distributed
5. **No Multicollinearity:** Predictors not highly correlated

Assumption Diagnostics

```r
# Diagnostic plots
par(mfrow = c(2, 2))
plot(model)

# Linearity (residuals vs fitted)
# Should show random scatter

# Normality (QQ plot)
# Points should follow diagonal line

# Homoscedasticity
# Residuals should have constant spread

# Normality test
shapiro.test(residuals(model))

# Multicollinearity (VIF)
library(car)
vif(model)
# VIF > 5 indicates problematic multicollinearity

# Durbin-Watson test (independence)
library(lmtest)
dwtest(model)
```

8.5 Logistic Regression

Logistic Regression Model

For binary outcomes:

$$\log\left(\frac{p}{1-p}\right) = \beta_0 + \beta_1 X_1 + \cdots + \beta_p X_p$$

$$p = \frac{1}{1 + e^{-(\beta_0 + \beta_1 X_1 + \cdots + \beta_p X_p)}}$$

Odds Ratio: $OR = e^{\beta_j}$

Logistic Regression in R

```r
# Fit logistic regression
model <- glm(outcome ~ x1 + x2,
             data = dataset,
             family = "binomial")

# View results
summary(model)

# Odds ratios
exp(coef(model))
exp(confint(model))

# Predictions (probabilities)
probs <- predict(model,
                 newdata = new_dataset,
                 type = "response")

# Predicted classes
classes <- ifelse(probs > 0.5, 1, 0)

# Model assessment
library(pROC)
roc_obj <- roc(dataset$outcome, probs)
auc(roc_obj)
```

8.6 Model Selection

Model Selection

```r
# Stepwise selection
model_full <- lm(y ~ ., data = dataset)
model_step <- step(model_full,
                   direction = "both")

# Best subsets
library(leaps)
regsubsets(y ~ ., data = dataset,
           nvmax = 10)

# Cross-validation
library(boot)
cv.glm(dataset, model, K = 10)

# LASSO regression
library(glmnet)
x <- model.matrix(y ~ ., dataset)[, -1]
y <- dataset$y
cv_fit <- cv.glmnet(x, y, alpha = 1)
plot(cv_fit)
```

8.7 Practice Exercises

1. Fit a simple linear regression model and interpret the coefficients.
2. Build a multiple regression model with interaction terms.
3. Check all regression assumptions and address violations.
4. Compare logistic regression models using AIC and AUC.
5. Perform model selection using stepwise methods.

9 Time Series Analysis and Forecasting

9.1 Introduction to Time Series

A time series is a sequence of observations collected at regular time intervals.

Time Series Components

- **Trend:** Long-term direction (increasing, decreasing, stable)
- **Seasonality:** Regular patterns within fixed periods
- **Cycle:** Fluctuations without fixed period
- **Irregular:** Random noise

9.2 Time Series Objects in R

Creating Time Series

```r
library(forecast)
library(tseries)

# Create time series object
ts_data <- ts(data_vector,
              start = c(2020, 1),
              frequency = 12)  # Monthly

# View
print(ts_data)
plot(ts_data)

# Convert to tsibble (modern approach)
library(tsibble)
tsibble_data <- tsibble(
    date = seq.Date(as.Date("2020-01-01"),
                    by = "month",
                    length.out = length(data_vector)),
    value = data_vector,
    index = date
)
```

9.3 Decomposition

Time Series Decomposition

```r
# Classical decomposition
decomp <- decompose(ts_data)
plot(decomp)

# STL decomposition (more robust)
decomp_stl <- stl(ts_data, s.window = "periodic")
plot(decomp_stl)

# Access components
decomp$trend
decomp$seasonal
decomp$random
```

9.4 Stationarity

Stationarity

A time series is stationary if:

- Mean is constant over time
- Variance is constant over time
- Autocovariance depends only on lag, not time

Augmented Dickey-Fuller Test:

$$H_0 : \text{Series has unit root (non-stationary)}$$

$$H_1 : \text{Series is stationary}$$

Stationarity Tests

```r
# ADF test
adf.test(ts_data)

# KPSS test (opposite hypotheses)
kpss.test(ts_data)

# If non-stationary, difference the series
ts_diff <- diff(ts_data)
adf.test(ts_diff)

# Visual inspection
plot(ts_data)
acf(ts_data)
```

9.5 Autocorrelation

ACF and PACF

```r
# Autocorrelation function
acf(ts_data, lag.max = 40)

# Partial autocorrelation function
pacf(ts_data, lag.max = 40)

# Combined plot
tsdisplay(ts_data)

# Ljung-Box test (autocorrelation in residuals)
Box.test(residuals,
         lag = 20,
         type = "Ljung-Box")
```

9.6 ARIMA Models

ARIMA(p

AR(p): Autoregressive of order p

$$X_t = c + \sum_{i=1}^{p} \phi_i X_{t-i} + \epsilon_t$$

I(d): Integrated of order d (differencing)
MA(q): Moving Average of order q

$$X_t = \mu + \epsilon_t + \sum_{i=1}^{q} \theta_i \epsilon_{t-i}$$

ARIMA Modeling

```r
library(forecast)

# Automatic ARIMA selection
model <- auto.arima(ts_data)

# Manual ARIMA
model <- Arima(ts_data,
               order = c(p, d, q))

# View model
summary(model)

# Diagnostics
checkresiduals(model)

# Forecast
forecast_values <- forecast(model, h = 12)
plot(forecast_values)

# Accuracy
accuracy(forecast_values, actual_values)
```

9.7 Exponential Smoothing

Exponential Smoothing

```r
# Simple exponential smoothing
model <- ses(ts_data, h = 12)

# Holt's linear trend
model <- holt(ts_data, h = 12)

# Holt-Winters (with seasonality)
model <- hw(ts_data,
            seasonal = "multiplicative",
            h = 12)

# ETS (automatic selection)
model <- ets(ts_data)
forecast_values <- forecast(model, h = 12)
plot(forecast_values)
```

9.8 Prophet

Facebook Prophet

```r
library(prophet)

# Prepare data
df <- data.frame(
    ds = dates,
    y = values
)

# Fit model
model <- prophet(df)

# Forecast
future <- make_future_dataframe(model,
                                periods = 365)
forecast <- predict(model, future)

# Plot
plot(model, forecast)
prophet_plot_components(model, forecast)
```

9.9 Model Evaluation

Forecast Accuracy

```r
# Split data
train <- window(ts_data, end = c(2023, 12))
test <- window(ts_data, start = c(2024, 1))

# Fit on train
model <- auto.arima(train)

# Forecast
fc <- forecast(model, h = length(test))

# Accuracy measures
accuracy(fc, test)

# Common metrics:
# ME, RMSE, MAE, MPE, MAPE, MASE, ACF1
```

9.10 Practice Exercises

1. Create a time series object and visualize the data.
2. Decompose a time series into trend, seasonal, and residual components.
3. Test for stationarity and apply transformations if needed.
4. Fit an ARIMA model and evaluate its performance.
5. Compare multiple forecasting methods on the same dataset.

10 Capstone Project

10.1 Project Overview

The capstone project integrates all skills learned throughout the module. Students will complete a comprehensive data analysis project from data acquisition to presentation of results.

Project Requirements

- Real-world dataset (minimum 1000 observations)
- Complete data cleaning and preprocessing
- Exploratory data analysis with visualizations
- Statistical analysis and modeling
- Written report (15-20 pages)
- Presentation (15 minutes)

10.2 Project Timeline

Week	Phase	Deliverable
1-2	Topic Selection & Data Collection	Project Proposal
3-4	Data Cleaning & EDA	Data Quality Report
5-6	Statistical Analysis	Analysis Code
7-8	Modeling & Validation	Model Results
9-10	Report Writing	Final Report
11	Presentation Preparation	Slide Deck
12	Final Presentation	Oral Defense

Table 5: Capstone Project Timeline

10.3 Project Structure

10.3.1 1. Introduction

- Background and context
- Problem statement
- Research questions
- Objectives

10.3.2 2. Literature Review

- Relevant previous work
- Theoretical framework
- Gap identification

10.3.3 3. Methodology

- Data source and description
- Data cleaning procedures
- Analytical methods
- Software and tools

10.3.4 4. Results

- Descriptive statistics
- Visualizations
- Statistical test results
- Model performance

10.3.5 5. Discussion

- Interpretation of findings
- Comparison with literature
- Limitations
- Implications

10.3.6 6. Conclusion

- Summary of key findings
- Recommendations
- Future work

10.4 Sample Project Topics

1. **Healthcare:** Predicting patient readmission rates using clinical data
2. **Finance:** Stock price forecasting using time series methods
3. **Marketing:** Customer segmentation for targeted campaigns
4. **Education:** Factors affecting student performance
5. **Environment:** Climate data analysis and trend detection
6. **Sports:** Player performance analysis and prediction
7. **Retail:** Sales forecasting and inventory optimization
8. **Social Media:** Sentiment analysis of user comments

10.5 Evaluation Criteria

Component	**Weight**
Data Quality & Preprocessing	20%
Exploratory Analysis	20%
Statistical Methods	25%
Modeling & Validation	20%
Report Quality	10%
Presentation	5%
Total	**100%**

Table 6: Capstone Project Evaluation

10.6 Report Guidelines

RMarkdown Template

```
---
title: "Capstone Project Report"
author: "Student Name"
date: "`r Sys.Date()`"
output:
  html_document:
    toc: true
    toc_float: true
    theme: cosmo
---

```{r setup, include=FALSE}
knitr::opts_chunk$set(echo = TRUE)
library(tidyverse)
```

Introduction

Background and problem statement...

Data

```{r data-import}
data <- read_csv("data.csv")
```

Analysis

```{r analysis}
# Your analysis code
```

Conclusion

Summary of findings...
```

10.7 Submission Requirements

1. **Report:** PDF format, 15-20 pages (excluding appendices)
2. **Code:** Well-commented R scripts or RMarkdown
3. **Data:** Clean dataset (or link to source)
4. **Presentation:** 10-15 slides

5. **GitHub Repository:** All materials version-controlled

10.8 Practice Exercises

1. Identify a real-world problem suitable for data analysis.
2. Find and document a dataset relevant to your chosen topic.
3. Create a project proposal with research questions and methodology.
4. Develop a timeline for completing all project phases.
5. Prepare a sample RMarkdown report structure.

A R Package Reference

Package	Purpose	Key Functions
`tidyverse`	Data manipulation	`dplyr`, `ggplot2`, `tidyr`
`readr`	Data import	`read_csv`, `read_tsv`
`readxl`	Excel files	`read_excel`
`dplyr`	Data wrangling	`filter`, `select`, `mutate`
`ggplot2`	Visualization	`ggplot`, `geom_point`
`car`	Regression diagnostics	`vif`, `leveneTest`
`forecast`	Time series	`auto.arima`, `forecast`
`lme4`	Mixed models	`lmer`, `glmer`
`pROC`	ROC analysis	`roc`, `auc`
`effsize`	Effect sizes	`cohen.d`

Table 7: Essential R Packages

B Statistical Tables

B.1 Critical Values for t-Distribution

df	0.10	0.05	0.025	0.01	0.005
1	3.078	6.314	12.706	31.821	63.657
5	1.476	2.015	2.571	3.365	4.032
10	1.372	1.812	2.228	2.764	3.169
20	1.325	1.725	2.086	2.528	2.845
30	1.310	1.697	2.042	2.457	2.750
∞	1.282	1.645	1.960	2.326	2.576

Table 8: t-Distribution Critical Values (Two-Tailed)

C Common R Errors and Solutions

Error	Solution
`object not found`	Check spelling, ensure object exists in environment
`could not find function`	Load required package with `library()`
`non-numeric argument`	Check data types with `str()`
`missing value where TRUE/FALSE needed`	Handle NA values before conditional
`subscript out of bounds`	Check vector/matrix dimensions
`replacement has length zero`	Ensure right-hand side has values

Table 9: Common R Errors

References

[1] Wickham, H., & Grolemund, G. (2019). *R for Data Science*. O'Reilly Media.

[2] James, G., Witten, D., Hastie, T., & Tibshirani, R. (2021). *An Introduction to Statistical Learning* (2nd ed.). Springer.

[3] Field, A., Miles, J., & Field, Z. (2018). *Discovering Statistics Using R*. Sage Publications.

[4] Hyndman, R. J., & Athanasopoulos, G. (2021). *Forecasting: Principles and Practice* (3rd ed.). OTexts.

[5] Wickham, H. (2023). *Advanced R* (2nd ed.). CRC Press.

[6] Navarro, D. (2023). *Learning Statistics with R*. University of Adelaide.

[7] McElreath, R. (2020). *Statistical Rethinking* (2nd ed.). CRC Press.

[8] Gelman, A., & Hill, J. (2020). *Data Analysis Using Regression and Multilevel/Hierarchical Models*. Cambridge University Press.